

Git & GitHub CLI Documentation

MacBook / VS Code Git Bash workflow used for Git identity, repository management, and bulk collaboration cleanup

1. Check the current Git repository

Use these commands to identify the connected GitHub repository and current branch.

```
git remote -v  
git remote get-url origin  
git branch --show-current  
git status
```

2. Check Git commit identity

These values control the author/committer identity stored in new commits.

```
git config user.name  
git config user.email  
git config --global user.name  
git config --global user.email
```

Set the repository-specific identity

```
git config user.name "Srabon"  
git config user.email "YOUR_GITHUB_EMAIL"
```

Use the email address associated with the intended GitHub account. The computer name shown in the terminal prompt (for example, Srabons-MacBook-M4) is the hostname, not the Git commit identity.

3. Inspect the latest commit identity

```
git show -s --format=fuller HEAD
```

4. Correct the latest commit author/committer

If the latest commit was created with the wrong identity and has not been safely shared, amend it after correcting user.name/user.email:

```
git commit --amend --reset-author --no-edit
```

If that amended commit replaces a commit already pushed to GitHub, the history has changed. A force push may be required, but use the safer form below and only when appropriate for the repository.

```
git push --force-with-lease
```

5. Handle divergent local and remote branches

When push is rejected with 'non-fast-forward' and pull reports divergent branches, a safe workflow is:

```
git fetch origin  
git pull --rebase origin main  
git status  
git log --oneline --decorate -5  
git push origin main
```

If Git reports a conflict, stop and resolve the conflict before continuing. Do not use `git reset --hard` or force-push blindly.

6. Install GitHub CLI (gh)

GitHub CLI is the command-line tool used to manage GitHub from the terminal. On macOS with Homebrew:

```
brew install gh
gh --version
```

7. Authenticate GitHub CLI

```
gh auth login
gh auth status
```

When logging in, select GitHub.com and HTTPS, then complete the browser/device authentication. Confirm the displayed account is the intended GitHub account.

8. List repositories

```
gh repo list --limit 1000
```

For more detailed repository data:

```
gh api user/repos --paginate --jq '.[ ] | "\(.full_name)\t\(.permissions.admin)\t\(.permissions.push)\t\(.permissions.pull)"'
```

Interpretation: admin=true generally indicates administrative ownership/control. A common collaboration case is admin=false, push=true, pull=true.

9. List only non-admin repositories

```
gh api user/repos --paginate --jq '.[ ] | select(.permissions.admin == false) | .full_name'
```

10. Leave a repository where someone else added you

This removes your collaborator access. It does not delete the repository.

```
gh api --method DELETE "/repos/OWNER/REPO/collaborators/srabon-chowdhury-47"
```

Example:

```
gh api --method DELETE "/repos/ajoy-dutta/example-repo/collaborators/srabon-chowdhury-47"
```

11. Bulk-leave a collaborator's repositories while keeping one

The workflow used in this session was to leave all ajoy-dutta repositories where the account is not an admin, except heavenauto.

Preview first:

```
gh api user/repos --paginate --jq '.[ ] | select(.owner.login == "ajoy-dutta" and .permissions.admin == false) | .full_name'
```

Execute only after reviewing the preview:

```
gh api user/repos --paginate --jq '.[ ] | select(.owner.login == "ajoy-dutta" and .permissions.admin == false) | .full_name' | \
  echo "Leaving $repo" | \
  gh api --method DELETE "/repos/$repo/collaborators/srabon-chowdhury-47" && echo done
```

This leaves ajoy-dutta/heavenauto untouched and does not delete any repositories.

12. Repository deletion vs. leaving a collaboration

Goal	Command	Effect
Leave collaborator repo	<code>gh api --method DELETE /repos/OWNER/REPO/collaborators/USERNAME</code>	Remove your access only
Delete your own repo	<code>gh repo delete OWNER/REPO --yes</code>	Permanently deletes the repository

If `gh repo delete` reports that the `delete_repo` scope is required:

```
gh auth refresh -h github.com -s delete_repo
```

Only use repository deletion when you intentionally want to permanently remove a repository you own.

13. Useful Git commands cheat sheet

Purpose	Command
Repository URL	<code>git remote -v</code>
Current branch	<code>git branch --show-current</code>
Working-tree status	<code>git status</code>
Recent commits	<code>git log --oneline -10</code>
Commit identity	<code>git config user.name / git config user.email</code>
Fetch remote updates	<code>git fetch origin</code>
Rebase onto remote main	<code>git pull --rebase origin main</code>
Push main	<code>git push origin main</code>

14. Safety checklist

- Preview bulk operations before executing them.
- Use collaborator-removal API calls when you want to leave a repository; do not use `gh repo delete`.
- Keep the repository you still need explicitly excluded from bulk commands.
- Do not force-push shared history unless you understand the impact.
- Use `git push --force-with-lease` instead of `--force` when rewriting history is necessary.
- Confirm `gh auth` status before performing account-level GitHub operations.